

Midterm – Creating Images with Diffusion Models

Contribution Journal

ITAI 2376 | Team DataProxy

Dataset: Fashion-MNIST

Team members

Elijah Raines • Brandon Matias • Justin Davis
Jonah Joseph • Fifehanmi Ogedengbe • Samantha Mireles

Project Overview

Our midterm uses Fashion-MNIST to build a diffusion model for clothing images. We begin with 28×28 grayscale images, add known random noise, and train a U-Net to predict that noise. Generation then starts with random noise and applies the reverse process over 100 steps while conditioning on a clothing class. The notebook connects setup, data preparation, model construction, diffusion, training, generation, visualization, and evaluation.

We divided the notebook by steps, but the sections share the same data, model, and tensor conventions. The captured training results show improving noise-prediction loss. Generation and evaluation are separate checks, so the loss history alone does not establish the quality of the final clothing images.

Jonah Joseph

Role(s)

Step 1: setup, tools, GPU checks, and reproducibility.

Key Contributions

Jonah handled the environment needed to run the notebook. He installed or imported PyTorch, torchvision, einops, and matplotlib, confirmed CUDA support, and checked that Colab had a Tesla T4 GPU available. He also set seed 42 and enabled deterministic CUDA settings so repeated debugging runs could be compared more reliably.

Evidence / Examples from Colab

The setup checks reported approximately 15.6 GB of GPU memory. That gave the team a concrete resource check before loading batches and training the model. PyTorch and torchvision supported the model and dataset work, einops supported tensor rearrangement, and matplotlib supported the image displays used in later steps.

Elijah Raines

Role(s)

Step 2: Fashion-MNIST preparation. Step 7: generation-process visualization and tensor-mask debugging.

Key Contributions

Elijah worked on the selected Option 2 configuration: 28×28 grayscale images, 10 classes, batch size 64, and a planned 30 epochs. He loaded the 60,000-sample dataset, checked GPU memory, and inspected image shape and pixel range. He created an 80/20 split with 48,000 training samples and 12,000 validation samples, then prepared both DataLoaders for the training loop.

In Step 7, Elijah worked on displaying selected stages of reverse denoising. He helped identify and correct a class-mask shape problem that interrupted the U-Net at a skip connection. This connected the visualization work back to the model's expected inputs.

Evidence / Examples from Colab

The PDF shows a sample image batch of [1, 1, 28, 28] with values in [-1, 1]. The full training and validation batches were [64, 1, 28, 28], with label shape [64]; the loaders contained 750 training batches and 188 validation batches. Step 7 failed with "Expected size 10 but got size 1" at `torch.cat((x, skip), dim=1)`. The follow-up discussion identified `torch.ones_like(c.unsqueeze(-1)).to(device)` as the correction for a [batch, 1] mask. The PDF still shows the old mask and error, so it does not document a successful rerun after this correction.

Samantha Mireles

Role(s)

Step 3: U-Net blocks, embeddings, and model assembly.

Key Contributions

Samantha handled GELUConvBlock, RearrangePoolBlock, DownBlock, and UpBlock, along with the sinusoidal time embedding and class EmbedBlock. She assembled the downsampling path, middle blocks, and upsampling path with skip connections. The model used channel dimensions (32, 64, 128), with time and class conditioning included in the denoising architecture.

Evidence / Examples from Colab

The named blocks show how the architecture was built in reusable pieces. Downsampling reduces spatial size as the channels increase; upsampling brings features back toward the image resolution. Skip connections carry earlier features into later blocks. Time conditioning supplies the diffusion stage, while class conditioning supplies the intended clothing category. Those inputs must remain compatible with the tensors passing through the network.

Brandon Matias

Role(s)

Step 4: diffusion functions and noise visualization. Step 6: class-conditioned generation and seed activity.

Key Contributions

Brandon worked on the forward diffusion equation, $x_t = \sqrt{\alpha_{\bar{t}}} * x_0 + \sqrt{1 - \alpha_{\bar{t}}} * \text{noise}$, and the reverse denoising function. The Step 4 reflection documents fixing “NameError: x_t is not defined.” Defining the noisy image correctly allowed the forward function to return the tensor needed by the rest of the pipeline.

He also worked on the noise-progression visualization and added bicubic interpolation for a cleaner display of the small 28×28 images. In Step 6, the generation function begins with random noise, conditions on a selected class, and calls reverse denoising through 100 steps. The section includes an activity for exploring different random seeds.

Evidence / Examples from Colab

The forward equation combines a scaled clean image with scaled random noise. Brandon’s reflection describes viewing the original image and stages labeled through 80% Noise. These labels identify positions in the schedule rather than measured pixel-noise percentages. Bicubic interpolation improves the displayed view without changing the dataset resolution. Step 6 contains the corrected [batch, 1] mask and the seed helper. The captured student activity remains blank; the follow-up chat supplies an Ankle boot example, rather than evidence of a completed output grid.

Fifehanmi Ogedengbe

Role(s)

Step 5: model training, validation, and training safeguards.

Key Contributions

Fifehanmi handled the training loop using Adam with learning rate 0.001 and mean squared error (MSE) loss to compare predicted noise with the noise actually added. The section includes gradient clipping, a scheduler, checkpoint saving, validation, early stopping logic, and sample generation. Before training, it checked the train and validation data for the expected [-1, 1] range and for NaN or infinite values.

Evidence / Examples from Colab

The model had 1,598,145 trainable parameters and an estimated 24.4 MB of model, gradient, and buffer usage. Data checks found no NaN or Inf values. Epoch 1 showed about 0.1873 training loss and 0.1529 validation loss; Epoch 2 showed about 0.1433 and 0.1382. Validation loss reached approximately 0.1145 by Epoch 10. The captured PDF continued into Epoch 11, so the evidence does not establish completion of all 30 planned epochs or that early stopping was triggered.

Justin Davis

Role(s)

Step 8: assigned CLIP evaluation section and evaluation setup.

Key Contributions

Justin's section sets up CLIP installation and dependencies for checking whether generated images match their intended classes. This adds an evaluation perspective beyond the training loss. The planned comparison connects the image produced by the generator with the clothing category requested during generation.

Evidence / Examples from Colab

The notebook assigns Step 8 to Justin and contains dependency installation, CLIP import, and available-model checks. The PDF has no visible setup-success output or final CLIP scores. Its explanatory comments still use digit descriptions from the starter notebook, so clothing-category prompts would need to be checked before evaluating Fashion-MNIST samples. His contribution is documented as the evaluation setup code; no completed scoring run or final category ranking is claimed.

For Fashion-MNIST, the evaluation question is whether an image requested as a particular clothing class visually agrees with that category. That is different from checking whether the model predicted the added noise accurately during training. Keeping these two checks separate helps the team describe what its results actually show.

Team Collaboration Summary

The notebook was divided by steps so each member had a clear part of the project. Jonah's setup supported Elijah's data preparation, and those batches became inputs to Samantha's U-Net and Brandon's diffusion functions. Fifehanmi's training used all of those pieces before the later generation, visualization, and evaluation sections could be checked.

The errors also crossed section boundaries. Brandon's undefined `x_t` interrupted the forward process that training depends on. Elijah's Step 7 `c_mask` mismatch appeared at a U-Net skip connection even though the input mask was created in the visualization cell. These examples showed why a shared notebook needs checks at the connections between steps, along with checks inside each member's section.

Our records show a working training process with improving loss and specific debugging work on diffusion and generation inputs. The captured run was still progressing, and the CLIP section had no final scores visible. Those limits matter when we describe the project's progress.

All team members should review this journal before final submission and confirm that the roles and examples match their actual work.